

COMUNICAÇÃO DISTRIBUÍDA EM SISTEMAS HETEROGÊNEOS UTILIZANDO ZEROMQ

2020032560 - BRUNO STANLLEY DE JESUS

2018016327 - LARISSA FAUSTINO FRANKLIN

2020028058 - PATRICKI HERNANDES DA COSTA

2020028352 - PATRICKY ALEXANDRE DA COSTA SILVA

SEMINÁRIO

Prof. Rafael Frinhani



INSTITUTO DE
MATEMÁTICA E
COMPUTAÇÃO

UNIFEI - Itajubá



Comunicação Distribuída em Sistemas Heterogêneos utilizando ZeroMQ

1 Introdução

O uso de sockets de baixo nível em sistemas distribuídos envolve desafios como implementação de protocolos, concorrência, troca não padronizada de mensagens e alto acoplamento entre componentes. Esses fatores impactam diretamente a escalabilidade e a manutenção dos sistemas, motivando o uso de middlewares como camada intermediária de abstração.

O middleware atua simplificando a comunicação entre aplicações distribuídas. Neste contexto, este seminário utiliza o ZeroMQ como tecnologia de estudo, com foco no paradigma *brokerless messaging*, no qual não há um intermediário central para roteamento de mensagens. Nesse modelo, os próprios nós se comunicam diretamente, reduzindo latência e aumentando a escalabilidade. Para validação prática, foram implementadas três operações utilizando a tecnologia.

O ZeroMQ é uma biblioteca de mensagens assíncronas de alto desempenho voltada para sistemas distribuídos e concorrentes. Ele abstrai sockets tradicionais, reduzindo a complexidade de comunicação em rede e aproximando-se da simplicidade de comunicação entre threads em um único processo [Pamadi et al. \(2020\)](#). Além disso, dispensa o uso de broker central e suporta diferentes padrões de comunicação e protocolos de transporte.

A tecnologia foi projetada para superar limitações de middlewares tradicionais, como baixa escalabilidade, alto consumo de recursos e dependência de intermediários centrais. Sua arquitetura favorece desempenho e eficiência por meio de técnicas como *zero-copy* [ZeroMQ Community \(2026\)](#).

A relevância deste estudo está na demanda por soluções escaláveis e de alto desempenho em sistemas distribuídos, especialmente em cenários que exigem baixo acoplamento entre componentes. Nesse contexto, o ZeroMQ se destaca como uma alternativa eficiente para comunicação distribuída.

2 Referencial Teórico

A comunicação em sistemas distribuídos é um dos elementos centrais desse tipo de arquitetura, ocorrendo por meio da troca de mensagens entre processos executados em diferentes máquinas [Tanenbaum & Van Steen \(2007\)](#). Esse modelo busca abstrair a complexidade da rede, permitindo maior escalabilidade, desacoplamento entre componentes e maior tolerância a falhas.

De forma geral, sistemas distribuídos são compostos por computadores independentes que operam de maneira coordenada, sendo percebidos pelo usuário como um único sistema coerente [Tanenbaum & Van Steen \(2007\)](#). Para viabilizar essa cooperação, diferentes modelos de comunicação são utilizados, entre eles:

- **RPC (Remote Procedure Call):** utilizado em interações cliente-servidor, abstraindo a troca de mensagens e simulando chamadas locais de função [Tanenbaum & Van Steen \(2007\)](#).
- **MOM (Message-Oriented Middleware):** baseado em mensagens assíncronas, adequado para cenários onde não há dependência estrita entre emissor e receptor.
- **Streaming:** voltado para transmissão contínua de dados sensíveis ao tempo.

2.1 MOM e paradigma Brokerless Messaging

O paradigma adotado neste seminário é o Brokerless Messaging, uma variação do MOM que elimina a necessidade de um broker central para o gerenciamento das mensagens. Nesse modelo, os próprios nós realizam a comunicação diretamente entre si, reduzindo intermediários.

Em um dos modelos do MOM, o baseado em broker, há um componente central responsável por receber, armazenar e encaminhar mensagens (como RabbitMQ e ActiveMQ). Já no modelo brokerless, essa função é eliminada, e a comunicação ocorre diretamente entre os processos, caracterizando uma arquitetura peer-to-peer. O ZeroMQ é um exemplo desse tipo de abordagem.

Um dos principais benefícios desse modelo é o desacoplamento entre os componentes:

- **Desacoplamento temporal:** emissor e receptor não precisam estar ativos simultaneamente.
- **Desacoplamento referencial:** os processos não precisam conhecer diretamente suas identidades, podendo se comunicar por canais ou tópicos.

De acordo com o estudo, a arquitetura brokerless também favorece o desempenho e a escalabilidade do sistema. Entre suas principais características estão:

- **Baixa latência:** comunicação direta entre processos reduz o tempo de resposta [Pamadi et al. \(2020\)](#).
- **Alto throughput:** capacidade de processar grandes volumes de mensagens simultaneamente.
- **Zero-copy:** otimização no uso de CPU e memória durante a transmissão de dados [Lauener & Sliwinski \(2017\)](#).
- **Escalabilidade horizontal:** ausência de ponto central de gargalo, permitindo crescimento do sistema sem degradação proporcional de desempenho.

2.2 Histórico do ZeroMQ

O ZeroMQ foi desenvolvido inicialmente por Pieter Hintjens e evoluiu com contribuição da comunidade. Sua adoção ganhou destaque em sistemas de alta demanda.

Segundo [Lauener & Sliwinski \(2017\)](#), um caso relevante ocorreu no CERN, onde o ZeroMQ foi adotado em 2011 para substituir uma solução baseada em CORBA, devido a limitações de escalabilidade e consumo de recursos.

A escolha se deu por sua arquitetura assíncrona, alto desempenho e suporte a técnicas como zero-copy, sendo posteriormente utilizado no controle de aceleradores do CERN.

3 Desenvolvimento

3.1 Arquitetura e funcionamento do ZeroMQ

Como mencionado, o ZeroMQ difere de modelos tradicionais de mensageria por não depender de um broker central. Ele é uma biblioteca de mensagens assíncronas que abstrai a complexidade de sockets, permitindo comunicação direta entre nós da aplicação e reduzindo a dependência de intermediários [Pamadi et al. \(2020\)](#).

3.1.1 Principais componentes

- **Contexto (Context):** estrutura responsável por gerenciar os sockets e threads de I/O utilizados na comunicação.
- **Sockets ZeroMQ:** abstraem detalhes de rede como reconexão, bufferização e enfileiramento de mensagens, simplificando a comunicação entre processos [Lauener & Sliwinski \(2017\)](#).
- **Serialização de mensagens:** a comunicação entre Python e Node.js ocorre por meio de mensagens convertidas em buffers de bytes. As mensagens são estruturadas em formato textual simples (ex.: opcao|payload), garantindo compatibilidade entre linguagens.

Embora seja possível utilizar formatos como JSON, optou-se por um protocolo textual simplificado para reduzir a complexidade da implementação.

A arquitetura do sistema pode ser representada por meio do modelo C4, em diferentes níveis de abstração.

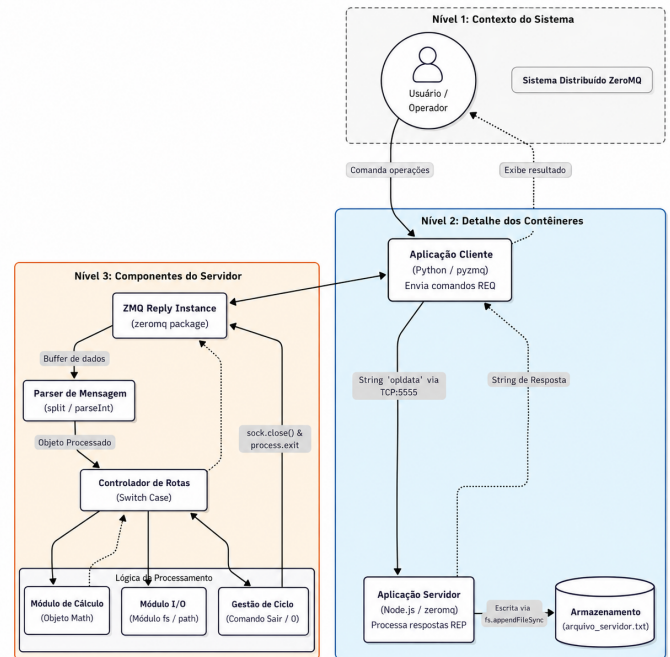


Figura 1: Diagrama C4 – Visão de Contêineres do ZeroMQ

3.1.2 Modelo de comunicação

O sistema adota comunicação assíncrona no nível do middleware, com o ZeroMQ realizando o transporte e enfileiramento das mensagens de forma não bloqueante [Lauener & Sliwinski \(2017\)](#).

No nível da aplicação, o padrão REQ/REP apresenta comportamento síncrono, pois o cliente envia uma requisição e aguarda a resposta antes de continuar. Assim, o ZeroMQ combina processamento assíncrono no transporte com interação síncrona na lógica da aplicação.

3.1.3 Padrões de interação

Os padrões de interação do ZeroMQ definem formas de comunicação entre nós distribuídos, encapsulando comportamentos comuns de rede e facilitando a construção de sistemas modulares e escaláveis. Os principais são:

- **Request/Reply:** modelo cliente-servidor em que uma requisição é enviada e uma resposta é retornada. Pode operar de forma síncrona ou assíncrona.
- **Publish/Subscribe:** distribuição de mensagens para múltiplos consumidores com base em tópicos.
- **Push/Pull:** utilizado para distribuição de tarefas em pipelines de processamento paralelo.

Neste projeto, foi utilizado o padrão Request/Reply, devido à necessidade de retorno imediato nas operações implementadas.

3.1.4 Modelo de consistência

A consistência do sistema é baseada no ordenamento FIFO dos sockets, garantindo que as mensagens sejam entregues ao servidor na mesma ordem em que foram enviadas pelo cliente.

Isso assegura previsibilidade no processamento das requisições.

3.2 Parte técnica e modelagem

3.2.1 Modelagem da solução e fluxo de comunicação

A solução foi estruturada separando a lógica de transporte da lógica de aplicação. O fluxo de comunicação ocorre nas seguintes etapas:

- O cliente monta a requisição no formato textual *opcao/payload*.
- O ZeroMQ realiza o transporte via TCP, convertendo os dados em buffers de bytes.
- O servidor recebe a mensagem, realiza o parsing e identifica a operação solicitada.
- A lógica correspondente é executada e o resultado é retornado ao cliente pelo padrão REQ/REP.

O fluxo completo do sistema pode ser representado por um diagrama BPMN, que descreve desde o envio da requisição até o retorno da resposta.

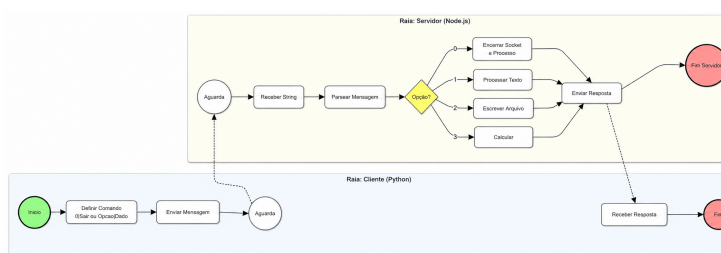


Figura 2: Diagrama BPMN do fluxo de comunicação do middleware

3.2.2 Bibliotecas utilizadas por componente

A implementação do sistema foi dividida entre cliente (Python) e servidor (Node.js), cada um utilizando bibliotecas específicas para suas responsabilidades.

Cliente (Python):

- **pyzmq**: Responsável pela comunicação com o servidor ZeroMQ, implementando o padrão Request (REQ).
- **argparse**: Biblioteca padrão do Python utilizada para leitura dos parâmetros de linha de comando (-host e -port).

Servidor (Node.js):

- **zeromq**: Biblioteca utilizada para implementação do socket Reply (REP) e gerenciamento da comunicação com o cliente.
- **fs**: Módulo responsável pela manipulação do arquivo .txt, permitindo a persistência dos dados.
- **Math**: Utilizado para execução das operações matemáticas requisitadas pelo cliente.

3.2.3 Parâmetros e configurações

Padrão de comunicação: Request-Reply (REQ-REP).

Protocolo de transporte: TCP.

Argumentos CLI (cliente):

- -host: endereço IP do servidor.
- -port: porta de conexão (ex: 5555).

Configuração do servidor:

- Bind no endpoint tcp://*:5555, permitindo conexões externas.

Formato das mensagens:

- Protocolo textual no formato OPCA0|PAYLOAD, onde a opção define a operação e o payload contém os dados.

O diagrama de implantação representa a distribuição dos componentes em ambiente distribuído, com cliente Python e servidor Node.js executando em máquinas distintas e comunicando-se via TCP através do ZeroMQ.

Também são destacadas as bibliotecas utilizadas (*pyzmq* e *zeromq*) e o endpoint de comunicação, evidenciando a integração entre tecnologias heterogêneas.

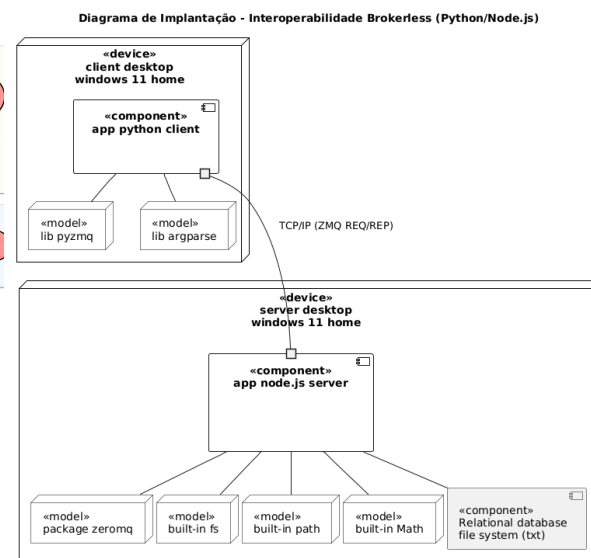


Figura 3: Diagrama de Implantação do Middleware Distribuído

3.3 Implementação de código

3.3.1 Servidor Node.js

A inicialização do servidor ocorre a partir da leitura dos parâmetros de entrada e da execução da função principal responsável pela configuração do socket. Em seguida, é criado um socket do tipo REP (Reply), definido o endpoint via TCP e realizada a operação de bind, que coloca o servidor em estado de escuta para requisições.

```
117 const argumentos = lerArgumentos();
118 iniciarServidor(argumentos.host, argumentos.port);
```

Figura 4: inicialização do servidor

```

27 async function iniciarServidor(host, port) {
28   const sock = new zmq.Reply();
29   const endpoint = `tcp://${host}:${port}`;
30   await sock.bind(endpoint);
31
32   console.log(`Servidor Node.js rodando em ${endpoint}...`);
33 }

```

Figura 5: bind / socket escutando

3.3.2 Cliente Python

A inicialização do cliente consiste na criação do contexto ZeroMQ, do socket do tipo REQ e na definição do endpoint TCP. Em seguida, o cliente se conecta ao servidor utilizando essas configurações, estabelecendo o canal de comunicação para envio de requisições e recebimento de respostas.

```

19 class ClienteZeroMQ:
20   def __init__(self, host:str, port:int):
21     self.host = host
22     self.port = port
23     self.context = zmq.Context().instance()
24     self.socket = self.context.socket(zmq.REQ)
25     self.endpoint = f"tcp://{self.host}:{self.port}"
26

```

Figura 6: cliente conectando

4 Demonstração prática

O diagrama de sequência a seguir apresenta o fluxo de comunicação entre cliente e servidor, destacando o envio da requisição, o processamento da mensagem e o retorno da resposta no padrão Request/Reply, conforme detalhado nas operações seguintes.

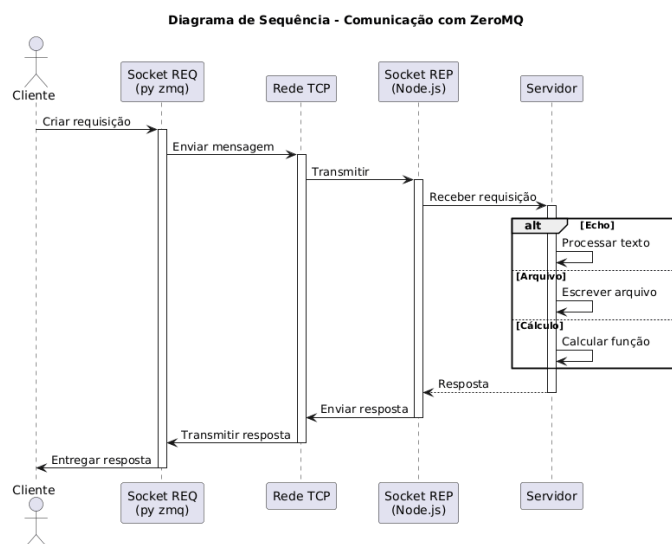


Figura 7: Diagrama de Sequência do Middleware Distribuído

4.1 Operação 1: Resposta a mensagem de texto

Esta operação envia uma string ao servidor. Se a entrada for "Hello", o servidor retorna "World"; caso contrário, retorna "Mensagem desconhecida".

```

Conectado ao servidor em tcp://192.168.3.39:5555

=== Menu do cliente ===
1 - Resposta a uma mensagem de texto
2 - Alterar um arquivo texto no servidor
3 - Calcular uma funcao
0 - Sair
Escolha uma opcao: 1
Digite a mensagem: Hello
Resposta do servidor: World

```

Figura 8: Resposta Hello World

4.2 Operação 2: Alteração do arquivo no servidor

Esta operação recebe um texto do cliente e o adiciona em modo *append* ao arquivo `arquivo_servidor.txt`, preservando seu conteúdo anterior no servidor.

```

=== Menu do cliente ===
1 - Resposta a uma mensagem de texto
2 - Alterar um arquivo texto no servidor
3 - Calcular uma funcao
0 - Sair
Escolha uma opcao: 2
Digite o texto para gravar no arquivo do servidor: teste
Resposta do servidor: Arquivo atualizado com sucesso.

=== Menu do cliente ===
1 - Resposta a uma mensagem de texto
2 - Alterar um arquivo texto no servidor
3 - Calcular uma funcao
0 - Sair
Escolha uma opcao: 2
Digite o texto para gravar no arquivo do servidor: zeromq
Resposta do servidor: Arquivo atualizado com sucesso.

```

Figura 9: Alteração do arquivo no servidor

4.3 Operação 3: Cálculo de funções

Esta operação realiza cálculos matemáticos no servidor, incluindo soma, potência, dobro, quadrado e raiz. O cliente envia a função e seus parâmetros no formato `funcao,param1,param2`.

```

=== Menu do cliente ===
1 - Resposta a uma mensagem de texto
2 - Alterar um arquivo texto no servidor
3 - Calcular uma funcao
0 - Sair
Escolha uma opcao: 3

Funcoes disponiveis: quadrado, dobro, soma, potencia, raiz
Digite a funcao e parametros (ex: quadrado,5): soma,5,6
Resposta do servidor: soma(5, 6) = 11

```

Figura 10: Execução do cálculo de funções

4.4 Demonstração da execução do servidor

A figura a seguir apresenta a execução do servidor em Node.js, demonstrando o recebimento das mensagens enviadas pelo cliente e o processamento das operações em tempo real.

```

larissa@frankling:~/Downloads/seminario-zeroMQ-main$ node server-node.js
Servidor Node.js rodando em tcp://*:5555...
Recebido: 1|Hello
Recebido: 1|ola
Recebido: 2|teste
Recebido: 2|zeromq
Recebido: 3|soma,5,6
Recebido: 3|quadrado,4

```

Figura 11: Demonstração da execução do servidor

5 Resultados

Nesta seção são apresentados os resultados obtidos a partir da execução dos experimentos realizados para validação da solução proposta na Seção *Desenvolvimento*. Os resultados correspondem às evidências práticas observadas durante a execução do sistema distribuído implementado com ZeroMQ.

5.1 Apresentação dos resultados experimentais

- **Operação 1: Resposta a mensagem de texto** Observou-se que o padrão Request/Reply (REQ/REP) permitiu comunicação direta entre aplicações em diferentes linguagens. O servidor em Node.js processou corretamente as mensagens enviadas pelo cliente em Python, evidenciando a abstração fornecida pelo ZeroMQ.
- **Operação 2: Alteração de arquivo no servidor** O cliente enviou uma solicitação de alteração de arquivo, executada e persistida corretamente no servidor. O ZeroMQ atuou como camada de transporte, enquanto o servidor realizou o acesso ao sistema de arquivos. O comportamento bloqueante do socket REQ garantiu a confirmação da operação antes de novas requisições.
- **Operação 3: Cálculo de funções** O servidor executou operações matemáticas como soma, raiz, quadrado e potência, demonstrando interoperabilidade entre Python e Node.js. Os dados foram corretamente interpretados, e a arquitetura brokerless contribuiu para menor latência ao eliminar intermediários.

5.2 Discussão dos Resultados

Os resultados indicam que o ZeroMQ contribuiu positivamente para a solução, principalmente na comunicação entre componentes distribuídos. No modelo brokerless, a ausência de um intermediário reduz a complexidade da infraestrutura e elimina um ponto único de falha. Nos testes, a comunicação ponto a ponto apresentou comportamento consistente e baixa latência em comparação conceitual com arquiteturas baseadas em brokers, embora dependa da disponibilidade entre cliente e servidor. Além disso, a interoperabilidade entre Python e Node.js foi viabilizada pelo uso de bibliotecas como pyzmq e zeromq.js, sendo suficiente o uso de um protocolo textual simples baseado em strings e bytes.

Apesar de suportar comunicação assíncrona, a implementação utilizou o padrão Request/Reply de forma bloqueante, o que favoreceu o controle das requisições e a estabilidade do sistema. De forma geral, os testes confirmam o atendimento aos objetivos propostos, com troca de mensagens, manipulação de arquivos e processamento matemático em ambiente distribuído. No entanto, observa-se que o modelo é mais adequado para cenários de baixa latência e mensagens leves, não sendo ideal para aplicações que exigem persistência garantida ou transferência de grandes volumes de dados.

6 Conclusões

Concluindo, este seminário apresentou a implementação e validação de um sistema distribuído baseado no ZeroMQ, com foco na comunicação entre sistemas heterogêneos. O objetivo foi viabilizar a troca de mensagens entre aplicações em diferentes linguagens, mantendo simplicidade, baixo acoplamento e eficiência.

A validação envolveu troca de mensagens de texto, manipulação de arquivos e cálculos matemáticos, demonstrando interoperabilidade entre linguagens e comportamento consistente do sistema. Os resultados indicam que os objetivos foram alcançados, com destaque para a eficiência da comunicação em ambiente distribuído.

Como contribuição, destaca-se a viabilidade do ZeroMQ em sistemas multi-linguagem. Como limitação, observa-se a ausência de persistência nativa e garantias fortes de entrega, o que restringe seu uso em aplicações críticas. Como trabalhos futuros, sugere-se a inclusão de mecanismos de tolerância a falhas e persistência de mensagens, além da comparação prática com outros middlewares, como RabbitMQ e Kafka.

7 Papéis e Responsabilidades

O desenvolvimento deste trabalho foi realizado de forma colaborativa entre todos os integrantes da equipe, com divisão de responsabilidades específicas para organização e execução das atividades.

7.1 Bruno Stanley

Responsável pela implementação do protótipo utilizando o middleware ZeroMQ, incluindo o desenvolvimento das aplicações cliente e servidor, configuração do ambiente, integração das bibliotecas e validação da comunicação distribuída. Também contribuiu com os testes do sistema, documentação técnica do código-fonte e evidências de execução do protótipo.

7.2 Larissa Faustino

Responsável pela coordenação e integração das atividades do grupo, incluindo planejamento, acompanhamento das tarefas, organização do seminário e comunicação com o docente. Também atuou na execução dos testes, demonstração prática do sistema, apresentação dos resultados, elaboração da conclusão e revisão geral de toda documentação.

7.3 Patricki Hernandez

Responsável pela definição da arquitetura do protótipo e modelagem da comunicação distribuída. Desenvolveu os diagramas arquiteturais, definiu os componentes do sistema e contribuiu para a fundamentação conceitual e documentação técnica da solução implementada.

7.4 Patricky Alexandre

Responsável pela análise da tecnologia e do paradigma de comunicação distribuída utilizados no seminário. Elaborou o referencial teórico, descreveu a arquitetura do middleware Ze-

roMQ, seus componentes, padrões de interação, vantagens, limitações e cenários de aplicação.

Por fim, todas as etapas do trabalho foram discutidas e validadas coletivamente pela equipe.

7.5 Documentação do Projeto

A documentação do projeto está disponível em: [Repositório no GitHub](#)

Referências

- Lauener, J. & Sliwinski, W. (2017). How to design & implement a modern communication middleware based on zeromq. Em *Proceedings of the 16th International Conference on Accelerator and Large Experimental Control Systems (ICALPCS)* Barcelona, Spain: JACoW Publishing. Acesso em: 20 abr. 2026.
- Pamadi, V. N., Goel, P., Himalayan, M. A., & Jain, A. (2020). Comparative analysis of grpc vs. zeromq for fast communication. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 7(2), 937–951. Acesso em: 20 abr. 2026.
- Tanenbaum, A. S. & Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Upper Saddle River, NJ: Pearson Prentice Hall, 2 edição. Acesso em: 20 abr. 2026.
- ZeroMQ Community (2026). Getting started with zeromq. Acesso em: 20 abr. 2026.

